

# Model Testing

```
In [7]: import pandas as pd
from tabulate import tabulate
import re

import os
os.environ['TF_USE_LEGACY_KERAS'] = '1'

import ktrain

import pickle

import warnings
warnings.filterwarnings('ignore')
```

```
In [8]: import spacy
nlp = spacy.load("en_core_web_sm")

def preprocess_and_tokenize(data):
    data = re.sub("<.*?>", "", data)
    data = re.sub(r'http\S+', '', data)
    data = re.sub(r"#[\d\w\.]+", '', data)
    data = re.sub(r"@[ \d\w\.]+", '', data)
    data = re.sub(r"(\W|\d)", " ", data)

    doc = nlp(data)

    processed_tokens = []

    for token in doc:
        if not token.is_punct and not token.is_space and \
            not token.is_digit and not token.is_stop and \
            len(token.text.strip()) > 1:
            processed_tokens.append(token.lemma_.lower())

    return processed_tokens
```

```
In [9]: def predict_emotion(text_message, model_path):
    """Loads the model and predicts emotion for a given text."""
    if model_path == 'models/bert_model':
        loaded_model = ktrain.load_predictor(model_path)
    else:
        with open(model_path, 'rb') as model_file:
            loaded_model = pickle.load(model_file)

    prediction = loaded_model.predict([text_message])
    probabilities = loaded_model.predict_proba([text_message])

    return prediction[0], probabilities[0]
```

## Traditional ML Models

```
In [10]: # Choose model with best preformance, which is Logistic Regression

logistic_results = []

message = 'he is a freak'

# You can also change to different model paths if you want to test,
# according to the saved models in 'models' folder.
emotion, confidence_scores = predict_emotion(message, 'models/tfidf_logistic_reg
max_confidence_score = max(confidence_scores)*100

logistic_results.append({'message': message, 'predicted_emotion': emotion, 'max_

print(f"Message: '{message}'")
print(f"Predicted emotion: {emotion}")
print(f"Confidence scores: {confidence_scores}")
print(f"Max confidence score (prediction): {max_confidence_score:.2f}%\n")

message = 'I cant decide what to do now, he is a freak'

emotion, confidence_scores = predict_emotion(message, 'models/tfidf_logistic_reg
max_confidence_score = max(confidence_scores)*100

logistic_results.append({'message': message, 'predicted_emotion': emotion, 'max_

print(f"Message: '{message}'")
print(f"Predicted emotion: {emotion}")
print(f"Confidence scores: {confidence_scores}")
print(f"Max confidence score (prediction): {max_confidence_score:.2f}%\n")
```

```
Message: 'he is a freak'
Predicted emotion: fear
Confidence scores: [0.0853835  0.51886327 0.08929051 0.22434702 0.0821157 ]
Max confidence score (prediction): 51.89%
```

```
Message: 'I cant decide what to do now, he is a freak'
Predicted emotion: fear
Confidence scores: [0.08721324 0.43274965 0.07698833 0.19523637 0.20781241]
Max confidence score (prediction): 43.27%
```

## Bert

```
In [11]: # BERT model that have been trained and saved already

bert_results = []

message = 'he is a freak'

emotion, confidence_scores = predict_emotion(message, 'models/bert_model')
max_confidence_score = max(confidence_scores)*100

bert_results.append({'message': message, 'predicted_emotion': emotion, 'max_conf

print(f"Message: '{message}'")
print(f"Predicted emotion: {emotion}")
print(f"Confidence scores: {confidence_scores}")
print(f"Max confidence score (prediction): {max_confidence_score:.2f}%\n")
```

```

message = 'I cant decide what to do now, he is a freak'

emotion, confidence_scores = predict_emotion(message, 'models/bert_model')
max_confidence_score = max(confidence_scores)*100

bert_results.append({'message': message, 'predicted_emotion': emotion, 'max_conf'

print(f"Message: '{message}'")
print(f"Predicted emotion: {emotion}")
print(f"Confidence scores: {confidence_scores}")
print(f"Max confidence score (prediction): {max_confidence_score:.2f}%\n")

```

Message: 'he is a freak'

Predicted emotion: neutral

Confidence scores: [0.00208904 0.01453433 0.06090649 0.01062401 0.91184616]

Max confidence score (prediction): 91.18%

Message: 'I cant decide what to do now, he is a freak'

Predicted emotion: fear

Confidence scores: [4.7199646e-05 2.4234962e-03 9.9295467e-01 1.0142121e-03 3.5604218e-03]

Max confidence score (prediction): 99.30%

## Short Conclusion

Based on the comparative analysis, the **BERT transformer model** demonstrates significantly superior performance over **traditional Logistic Regression** for emotion classification.

BERT shows much higher prediction confidence (91.18-99.30% vs. 43.27-51.89%) and better contextual understanding.

For "he is a freak," Logistic Regression incorrectly predicts "**fear**" with low confidence, while BERT correctly identifies "**neutral**" context with high certainty. Both models agree on the extended sentence, but BERT maintains exceptional confidence.

The substantial confidence gap and BERT's nuanced semantic processing highlight transformer models' advantage in capturing emotional subtleties and contextual meaning beyond traditional bag-of-words approaches.

```

In [15]: summary_stats = []

for i in range(len(logistic_results)):
    log_max = logistic_results[i]['max_confidence']
    bert_max = bert_results[i]['max_confidence']
    confidence_gap = bert_max - log_max

    summary_stats.append({
        'Message': logistic_results[i]['message'][:18] + "...",
        'LR_Pred': logistic_results[i]['predicted_emotion'],
        'BERT_Pred': bert_results[i]['predicted_emotion'],
        'LR_Confi': f"{log_max:.2f}%",
        'BERT_Confi': f"{bert_max:.2f}%",
        'Confi_Diff': f"{confidence_gap:+.2f}%",
    })

```

```

        'Same_Pred': 'Yes' if logistic_results[i]['predicted_emotion'] == bert_r
    })

summary_df = pd.DataFrame(summary_stats)
print("")
print(tabulate(summary_df, headers='keys', tablefmt='grid', showindex=False))

print(f"\nAverage Confidence Scores:")
print(f"  Logistic Regression: {(logistic_results[0]['max_confidence'] + logisti
print(f"  BERT Transformer: {(bert_results[0]['max_confidence'] + bert_results[1
print(f"\nPerformance Improvement: {(bert_results[0]['max_confidence'] + bert_r

```

```

+-----+-----+-----+-----+-----+-----+
+-----+-----+
| Message          | LR_Pred  | BERT_Pred  | LR_Confi  | BERT_Confi  | C
onfi_Diff  | Same_Pred  |
+=====+=====+=====+=====+=====+=====+
=====+=====+
| he is a freak    | fear     | neutral    | 51.89%    | 91.18%      | +
39.30%    | No       |
+-----+-----+-----+-----+-----+-----+
+-----+-----+
| I cant decide what... | fear     | fear       | 43.27%    | 99.30%      | +
56.02%    | Yes      |
+-----+-----+-----+-----+-----+-----+
+-----+-----+

```

Average Confidence Scores:

Logistic Regression: 47.58%

BERT Transformer: 95.24%

Performance Improvement: 47.66%